

Vehicle Damage Detection

Proof of Concept (POC)

Client: VROOM Cars

Project by: Tanmay Pakori

Project Overview

Manual vehicle damage inspection is often time-consuming, inconsistent, and dependent on human expertise. This project develops a **Proof of Concept (POC)** for an AI-powered vehicle damage detection system that classifies front and rear vehicle images into six predefined categories using deep learning. The solution is integrated with a Streamlit application to evaluate the feasibility of automated damage classification.

Project Objective

To evaluate the feasibility of using deep learning for automated vehicle damage classification and identify the most suitable model for future production deployment.

Problem Statement

Develop a Proof of Concept (POC) to classify uploaded vehicle images into six predefined damage categories with a target validation accuracy of **75% or higher**.

Damage Categories

1. Front Normal
2. Front Breakage
3. Front Crushed
4. Rear Normal
5. Rear Breakage
6. Rear Crushed

Approach

- Data preprocessing & augmentation
- Baseline CNN model development
- Transfer learning using EfficientNet-B0 and ResNet-50
- Hyperparameter tuning using Optuna
- Model evaluation and comparison
- Final model selection based on validation accuracy

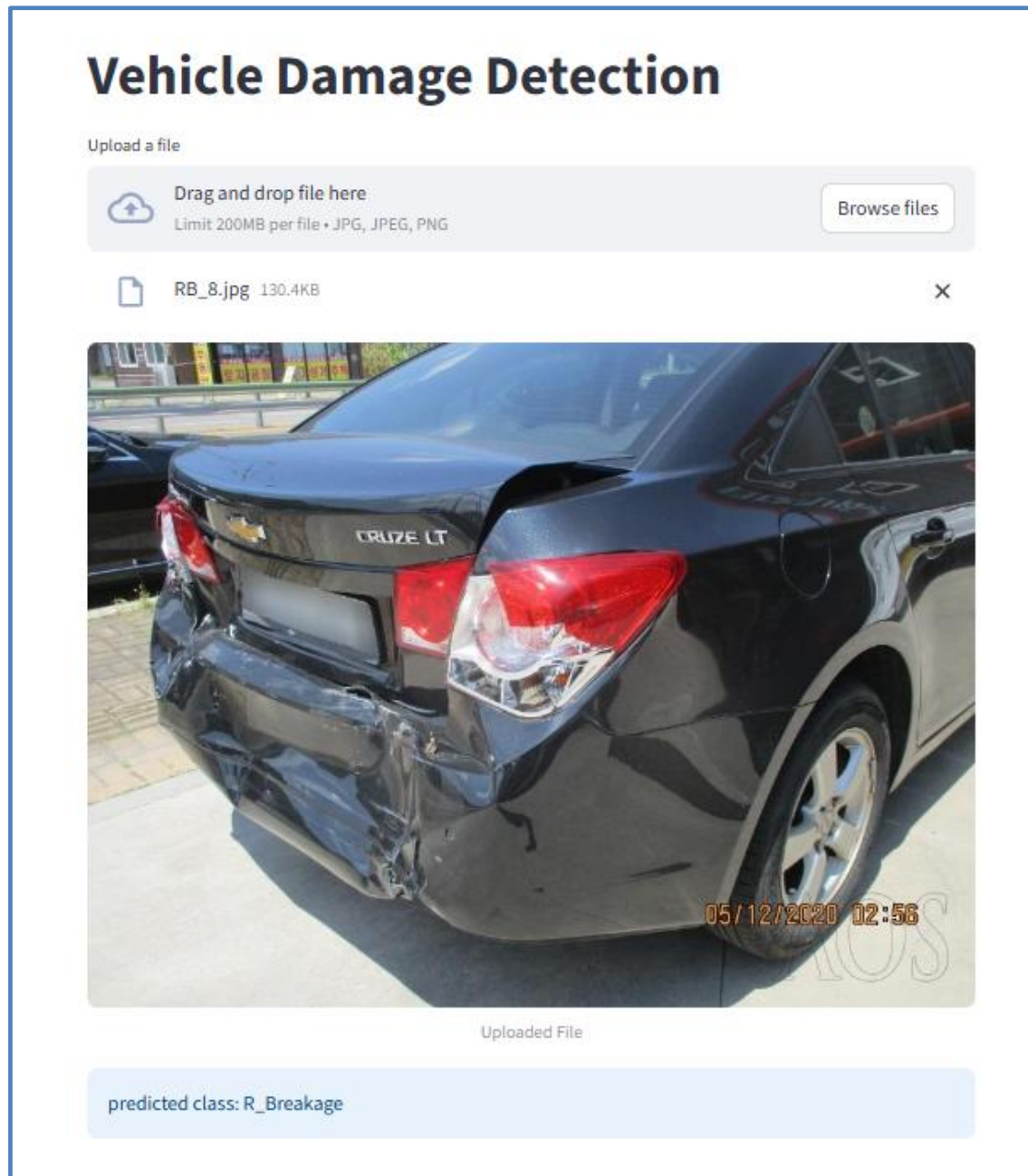
Tech Stack

Python, PyTorch, Torchvision, Optuna, Streamlit, NumPy, Matplotlib

Deliverables

- Trained deep learning model with source code
- Target Accuracy: >75%
- Streamlit application for drag-and-drop image prediction

Streamlit app interface





AI-Powered Vehicle Damage Detection

Deep Learning-Based Car Damage Classification using CNN, EfficientNet-B0 & ResNet-50

Project by: Tanmay Pakori

Python 3.11 PyTorch Deep Learning Transfer Learning EfficientNet | ResNet Status Completed

```
In [2]: import torch
import torch_directml
import pandas as pd
from sklearn.model_selection import train_test_split
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from matplotlib import pyplot as plt
from torchvision import datasets, transforms
import torchvision
from torchvision import models
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights
from torch.nn import BatchNorm2d
import torch.nn.functional as F
import time
import os
```

Create a DirectML device to use the GPU (AMD/Intel/DirectX-supported) for PyTorch computations.

```
In [3]: device = torch_directml.device()
device
```

```
Out[3]: device(type='privateuseone', index=0)
```

Load Data

Create an image preprocessing pipeline that applies data augmentation (horizontal flip and color adjustments), resizes images to 224×224, converts them into PyTorch tensors, and normalizes pixel values using ImageNet statistics for EfficientNet training.

```
In [4]: image_transforms=transforms.Compose([
    transforms.RandomHorizontalFlip(10),
    transforms.ColorJitter(brightness=0.2,contrast=0.2),
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406],std=[0.229,0.224,0.225])
])
```

Load the image dataset from the specified directory, apply the preprocessing transformations, and return the total number of images in the dataset.

```
In [5]: dataset_path = "./dataset"

dataset = datasets.ImageFolder(root=dataset_path,transform=image_transforms)
len(dataset)
```

```
Out[5]: 2300
```

```
In [24]: class_names = dataset.classes
class_names
```

```
Out[24]: ['F_Breakage', 'F_Crushed', 'F_Normal', 'R_Breakage', 'R_Crushed', 'R_Normal']
```

```
In [7]: num_classes = len(dataset.classes)
num_classes
```

```
Out[7]: 6
```

```
In [8]: train_size = int(0.75*len(dataset))
val_size = len(dataset)-train_size
train_size,val_size
```

```
Out[8]: (1725, 575)
```

Split the dataset into training and validation subsets for model training and performance evaluation.

```
In [9]: from torch.utils.data import random_split

train_dataset,val_dataset = random_split(dataset,[train_size,val_size])
```

Create DataLoaders to efficiently load training and validation data in batches, with shuffling enabled for random sampling during training.

```
In [10]: train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=True)
```

Retrieve one batch of images and labels from the training DataLoader to verify the batch size and data dimensions.

Retrieve one batch of images and labels from the training DataLoader to verify the batch size and data dimensions.

```
In [11]: for images,labels in train_loader:
    print(images.shape)
```

```
print(labels.shape)
break
```

```
torch.Size([32, 3, 224, 224])
torch.Size([32])
```

`torch.Size([32, 3, 224, 224])` → A batch of 32 RGB images, where each image has 3 color channels (RGB) and a size of 224×224 pixels. `torch.Size([32])` → A batch of 32 labels, with one class label corresponding to each image in the batch.

```
In [11]: labels[0]
```

```
Out[11]: tensor(5)
```

```
In [12]: images[0].shape
```

```
Out[12]: torch.Size([3, 224, 224])
```

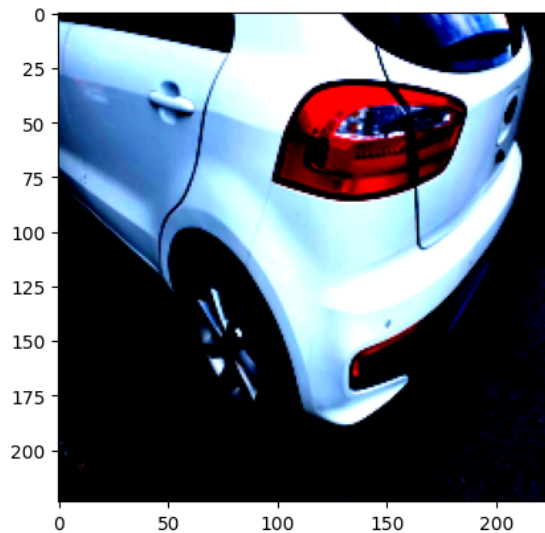
```
In [13]: #reshaping to plot using imshow
images[0].permute(1,2,0).shape
```

```
Out[13]: torch.Size([224, 224, 3])
```

Display the first image from the training batch by rearranging the tensor dimensions from (Channels, Height, Width) to (Height, Width, Channels) for visualization.

```
In [14]: plt.imshow(images[0].permute(1,2,0))
plt.show()
```

Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-2.0665298..2.0299783].



Model 1

Define a Convolutional Neural Network (CNN) for car image classification using multiple convolutional, ReLU activation, and max-pooling layers to extract features, followed by fully connected layers that classify the extracted features into the specified number of car damage classes.

```
In [15]: class CarClassifierCNN(nn.Module):
def __init__(self,num_classes):
    super().__init__()
    self.network=nn.Sequential(
        nn.Conv2d(in_channels=3,out_channels=16,kernel_size=3,stride=1,padding=1), #(16,224,224)
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2,stride=2,padding=0), #(16,112,112)
        nn.Conv2d(in_channels=16,out_channels=32,kernel_size=3,stride=1,padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2,stride=2,padding=0), #(32,56,56)
        nn.Conv2d(in_channels=32,out_channels=64,kernel_size=3,stride=1,padding=1),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2,stride=2,padding=0), #(64,28,28)

        nn.Flatten(),
        nn.Linear(64*28*28,512),
        nn.ReLU(),

        nn.Linear(512,num_classes)

    )
def forward(self,X):
    X = self.network(X)
    return X
```

Initialize the CNN model on the selected device, define the cross-entropy loss function for multi-class classification, and configure the Adam optimizer to update the model's parameters during training.

```
In [16]: model = CarClassifierCNN(num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(),lr=0.001)
```

Define the model training function that performs forward and backward propagation, updates model parameters using the optimizer, calculates training loss, evaluates validation accuracy after each epoch, records predictions for performance analysis, and measures the total training execution time.

```
In [16]: def train_model(model,criterion,optimizer,epochs=5):
    start=time.time()
    for epoch in range(epochs):
        model.train()
        running_loss=0.0
        for batch_num, (images,labels) in enumerate(train_loader):
            images,labels=images.to(device), labels.to(device)

            #forward pass
            outputs=model(images)
            #calculate loss
            loss=criterion(outputs,labels)
            #backpass
            optimizer.zero_grad()
            loss.backward()
            #update grads
            optimizer.step()

            if (batch_num+1)%10==0:
                print(f"Batch:{batch_num+1}, Loss:{loss.item():0.2f}")

            running_loss+=loss.item()*images.size(0)
        epoch_loss=running_loss/len(train_loader.dataset)
        print(f"Epoch:[{epoch+1}/{epochs}], Avg Loss:{epoch_loss:.4f}")

        #validation
        model.eval()
        total = 0
        correct=0
        all_labels=[]
        all_predictions=[]

        with torch.no_grad():
            for images,labels in val_loader:
                images,labels=images.to(device), labels.to(device)
                outputs=model(images)
                _, predicted = torch.max(outputs,1)
                total +=labels.size(0)
                correct += (predicted==labels).sum().item()
                all_labels.extend(labels.cpu().numpy())
                all_predictions.extend(predicted.cpu().numpy())

            print(f"*** Validation Accuracy: {100*correct/total:.2f}%***")

        end = time.time()
        print(f"Execution_time: {end-start} seconds")
    return all_labels,all_predictions
```

```
In [18]: all_labels,all_predictions = train_model(model,criterion,optimizer,epochs=10)
```

```
C:\Users\tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\optim\adam.py:534: UserWarning: The operator 'aten::lerp.Scalar_out' is not currently supported on the DML backend and will fall back to run on the CPU. This may have performance implications. (Triggered internally at C:\__w\1\pytorch-directml-plugin\torch_directml\csrc\dml\dml_cpu_fallback.cpp:17.)
  torch._foreach_lerp_(device_exp_avgs, device_grads, 1 - beta1)
```

```

Batch:10, Loss:1.79
Batch:20, Loss:1.77
Batch:30, Loss:1.48
Batch:40, Loss:1.30
Batch:50, Loss:1.35
Epoch:[1/10], Avg Loss:1.6170
*** Validation Accuracy: 46.43%***
Execution_time: 120.9537250995636 seconds
Batch:10, Loss:1.23
Batch:20, Loss:0.96
Batch:30, Loss:0.93
Batch:40, Loss:1.20
Batch:50, Loss:0.99
Epoch:[2/10], Avg Loss:1.1224
*** Validation Accuracy: 48.35%***
Execution_time: 226.38715195655823 seconds
Batch:10, Loss:0.78
Batch:20, Loss:0.79
Batch:30, Loss:0.90
Batch:40, Loss:0.73
Batch:50, Loss:0.52
Epoch:[3/10], Avg Loss:0.8300
*** Validation Accuracy: 52.35%***
Execution_time: 328.54786133766174 seconds
Batch:10, Loss:0.50
Batch:20, Loss:0.39
Batch:30, Loss:0.57
Batch:40, Loss:0.66
Batch:50, Loss:0.56
Epoch:[4/10], Avg Loss:0.5263
*** Validation Accuracy: 56.87%***
Execution_time: 431.6210722923279 seconds
Batch:10, Loss:0.26
Batch:20, Loss:0.08
Batch:30, Loss:0.24
Batch:40, Loss:0.11
Batch:50, Loss:0.12
Epoch:[5/10], Avg Loss:0.2278
*** Validation Accuracy: 55.65%***
Execution_time: 534.3427948951721 seconds
Batch:10, Loss:0.12
Batch:20, Loss:0.06
Batch:30, Loss:0.03
Batch:40, Loss:0.03
Batch:50, Loss:0.13
Epoch:[6/10], Avg Loss:0.0929
*** Validation Accuracy: 52.70%***
Execution_time: 637.9634249210358 seconds
Batch:10, Loss:0.04
Batch:20, Loss:0.28
Batch:30, Loss:0.02
Batch:40, Loss:0.01
Batch:50, Loss:0.02
Epoch:[7/10], Avg Loss:0.0348
*** Validation Accuracy: 53.74%***
Execution_time: 740.6022706031799 seconds
Batch:10, Loss:0.01
Batch:20, Loss:0.00
Batch:30, Loss:0.01
Batch:40, Loss:0.00
Batch:50, Loss:0.00
Epoch:[8/10], Avg Loss:0.0186
*** Validation Accuracy: 54.43%***
Execution_time: 843.9982914924622 seconds
Batch:10, Loss:0.00
Batch:20, Loss:0.00
Batch:30, Loss:0.00
Batch:40, Loss:0.00
Batch:50, Loss:0.03
Epoch:[9/10], Avg Loss:0.0121
*** Validation Accuracy: 51.30%***
Execution_time: 946.6515011787415 seconds
Batch:10, Loss:0.01
Batch:20, Loss:0.01
Batch:30, Loss:0.00
Batch:40, Loss:0.00
Batch:50, Loss:0.00
Epoch:[10/10], Avg Loss:0.0213
*** Validation Accuracy: 53.57%***
Execution_time: 1050.6689121723175 seconds

```

Model 2 CNN with Regularization

Define a CNN with Batch Normalization and Dropout regularization to improve training stability, reduce overfitting, and classify car images into the specified number of classes.

```

In [19]: class CarClassifierCNNwithRegularization(nn.Module):
          def __init__(self,num_classes):
              super().__init__()
              self.network=nn.Sequential(
                  nn.Conv2d(in_channels=3,out_channels=16,kernel_size=3,stride=1,padding=1), #(16,224,224)
                  BatchNorm2d(16),
                  nn.ReLU(),
                  nn.MaxPool2d(kernel_size=2,stride=2,padding=0), #(16,112,112)
                  nn.Conv2d(in_channels=16,out_channels=32,kernel_size=3,stride=1,padding=1),
                  BatchNorm2d(32),
                  nn.ReLU(),

```

```

        nn.MaxPool2d(kernel_size=2, stride=2, padding=0), #(32, 56, 56)
        nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3, stride=1, padding=1),
        BatchNorm2d(64),
        nn.ReLU(),
        nn.MaxPool2d(kernel_size=2, stride=2, padding=0), #(64, 28, 28)

        nn.Flatten(),
        nn.Linear(64*28*28, 512),
        nn.ReLU(),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)

    )
    def forward(self, X):
        X = self.network(X)
        return X

```

Initialize the regularized CNN model, define the loss function and Adam optimizer with L2 weight decay, then train the model for 10 epochs while recording validation labels and predictions.

```

In [20]: model = CarClassifierCNNwithRegularization(num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

all_labels, all_predictions = train_model(model, criterion, optimizer, epochs=10)

```

```

Batch:10, Loss:11.39
Batch:20, Loss:5.38
Batch:30, Loss:2.78
Batch:40, Loss:1.40
Batch:50, Loss:1.53
Epoch:[1/10], Avg Loss:7.0225
*** Validation Accuracy: 48.52%***
Execution_time: 110.13337397575378 seconds
Batch:10, Loss:1.46
Batch:20, Loss:1.42
Batch:30, Loss:1.20
Batch:40, Loss:1.28
Batch:50, Loss:1.45
Epoch:[2/10], Avg Loss:1.2561
*** Validation Accuracy: 53.91%***
Execution_time: 220.01667284965515 seconds
Batch:10, Loss:0.96
Batch:20, Loss:1.07
Batch:30, Loss:1.25
Batch:40, Loss:1.03
Batch:50, Loss:1.28
Epoch:[3/10], Avg Loss:1.1454
*** Validation Accuracy: 50.43%***
Execution_time: 329.50338530540466 seconds
Batch:10, Loss:1.03
Batch:20, Loss:0.99
Batch:30, Loss:1.12
Batch:40, Loss:1.03
Batch:50, Loss:0.82
Epoch:[4/10], Avg Loss:1.0544
*** Validation Accuracy: 55.13%***
Execution_time: 443.8637595176697 seconds
Batch:10, Loss:0.98
Batch:20, Loss:0.86
Batch:30, Loss:1.18
Batch:40, Loss:1.24
Batch:50, Loss:1.11
Epoch:[5/10], Avg Loss:1.0246
*** Validation Accuracy: 54.26%***
Execution_time: 552.8810317516327 seconds
Batch:10, Loss:0.86
Batch:20, Loss:0.93
Batch:30, Loss:0.76
Batch:40, Loss:1.13
Batch:50, Loss:0.90
Epoch:[6/10], Avg Loss:1.0005
*** Validation Accuracy: 56.17%***
Execution_time: 660.7917549610138 seconds
Batch:10, Loss:1.20
Batch:20, Loss:1.10
Batch:30, Loss:1.07
Batch:40, Loss:0.84
Batch:50, Loss:1.04
Epoch:[7/10], Avg Loss:0.9574
*** Validation Accuracy: 59.13%***
Execution_time: 769.6878385543823 seconds
Batch:10, Loss:0.69
Batch:20, Loss:0.60
Batch:30, Loss:0.72
Batch:40, Loss:1.00
Batch:50, Loss:0.96
Epoch:[8/10], Avg Loss:0.8931
*** Validation Accuracy: 60.00%***
Execution_time: 877.6547796726227 seconds
Batch:10, Loss:0.94
Batch:20, Loss:0.88
Batch:30, Loss:0.87
Batch:40, Loss:0.95
Batch:50, Loss:1.11
Epoch:[9/10], Avg Loss:0.9189
*** Validation Accuracy: 58.78%***
Execution_time: 986.1332771778107 seconds
Batch:10, Loss:1.00
Batch:20, Loss:1.01
Batch:30, Loss:1.32
Batch:40, Loss:0.57
Batch:50, Loss:0.84
Epoch:[10/10], Avg Loss:0.8802
*** Validation Accuracy: 58.61%***
Execution_time: 1094.806982755661 seconds

```

Model 3 Transfer Learning with EfficientNet

Load the pretrained EfficientNet-B0 model and retrieve the number of input features to its final classification layer for customization.

```
In [21]: model = models.efficientnet_b0(weights='DEFAULT')
        model.classifier[1].in_features
```

```
Out[21]: 1280
```

Define a transfer learning model using a pretrained EfficientNet-B0, freeze its feature extraction layers, replace the final classifier with a custom dropout and fully connected layer, and adapt it for the specified number of output classes.

```
In [26]: class CarClassifierEfficientNet(nn.Module):
        def __init__(self, num_classes):
            super().__init__()
```



```

self.model=models.efficientnet_b0(weights='DEFAULT')

for param in self.model.parameters():
    param.requires_grad=False

in_features=self.model.classifier[1].in_features

self.model.classifier=nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(in_features,num_classes)
)
def forward(self,X):
    X = self.model(X)
    return X

```

Initialize the EfficientNet transfer learning model, define the loss function, optimize only the trainable classifier parameters using Adam, and train the model for 10 epochs while collecting validation labels and predictions.

```

In [28]: model = CarClassifierEfficientNet(num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad,model.parameters()),lr=0.001)

all_labels,all_predictions=train_model(model,criterion,optimizer,epochs=10)

```

```

Batch:10, Loss:1.67
Batch:20, Loss:1.54
Batch:30, Loss:1.36
Batch:40, Loss:1.31
Batch:50, Loss:1.37
Epoch:[1/10], Avg Loss:1.4666
*** Validation Accuracy: 60.17%***
Execution_time: 123.55034756660461 seconds
Batch:10, Loss:1.30
Batch:20, Loss:0.96
Batch:30, Loss:0.97
Batch:40, Loss:1.18
Batch:50, Loss:0.94
Epoch:[2/10], Avg Loss:1.1229
*** Validation Accuracy: 64.35%***
Execution_time: 241.9060480594635 seconds
Batch:10, Loss:0.87
Batch:20, Loss:0.88
Batch:30, Loss:1.18
Batch:40, Loss:0.98
Batch:50, Loss:1.02
Epoch:[3/10], Avg Loss:0.9905
*** Validation Accuracy: 65.91%***
Execution_time: 356.78325510025024 seconds
Batch:10, Loss:1.00
Batch:20, Loss:1.17
Batch:30, Loss:0.92
Batch:40, Loss:0.95
Batch:50, Loss:1.03
Epoch:[4/10], Avg Loss:0.9353
*** Validation Accuracy: 66.43%***
Execution_time: 476.3303325176239 seconds
Batch:10, Loss:0.99
Batch:20, Loss:0.72
Batch:30, Loss:0.90
Batch:40, Loss:0.95
Batch:50, Loss:0.98
Epoch:[5/10], Avg Loss:0.8844
*** Validation Accuracy: 68.35%***
Execution_time: 591.8674001693726 seconds
Batch:10, Loss:0.99
Batch:20, Loss:0.77
Batch:30, Loss:1.07
Batch:40, Loss:0.74
Batch:50, Loss:0.69
Epoch:[6/10], Avg Loss:0.8396
*** Validation Accuracy: 66.96%***
Execution_time: 705.8958461284637 seconds
Batch:10, Loss:0.74
Batch:20, Loss:0.82
Batch:30, Loss:0.73
Batch:40, Loss:0.74
Batch:50, Loss:0.78
Epoch:[7/10], Avg Loss:0.8070
*** Validation Accuracy: 65.57%***
Execution_time: 819.3766205310822 seconds
Batch:10, Loss:0.75
Batch:20, Loss:0.73
Batch:30, Loss:0.80
Batch:40, Loss:0.62
Batch:50, Loss:0.58
Epoch:[8/10], Avg Loss:0.7655
*** Validation Accuracy: 66.26%***
Execution_time: 934.3250572681427 seconds
Batch:10, Loss:0.81
Batch:20, Loss:0.94
Batch:30, Loss:0.73
Batch:40, Loss:0.77
Batch:50, Loss:0.79
Epoch:[9/10], Avg Loss:0.7922
*** Validation Accuracy: 67.83%***
Execution_time: 1050.3484721183777 seconds
Batch:10, Loss:0.87
Batch:20, Loss:0.56
Batch:30, Loss:0.91
Batch:40, Loss:0.86
Batch:50, Loss:0.71
Epoch:[10/10], Avg Loss:0.7591
*** Validation Accuracy: 67.48%***
Execution_time: 1165.1470041275024 seconds

```

Model 4 Transfer Learning with ResNet

Define a transfer learning model using a pretrained ResNet-50 by freezing most layers, fine-tuning the final feature extraction block (layer4), replacing the classification head with a custom dropout and fully connected layer, and adapting it for the specified number of output classes.

```

In [17]: class CarClassifierResNet(nn.Module):
          def __init__(self, num_classes, dropout_rate=0.5):
              super().__init__()
              self.model=models.resnet50(weights='DEFAULT')
              # Freeze all layers except the final fully connected layer
              for param in self.model.parameters():
                  param.requires_grad=False

              # Unfreeze layer4 and fully connected layer
              for param in self.model.layer4.parameters():
                  param.requires_grad=True

```

```

# Replace the final fully connected layer
self.model.fc=nn.Sequential(
    nn.Dropout(0.5),
    nn.Linear(self.model.fc.in_features,num_classes)
)
def forward(self,X):
    X = self.model(X)
    return X

```

Initialize the ResNet-50 transfer learning model, define the cross-entropy loss function, optimize only the trainable (unfrozen) parameters using Adam, and train the model for 10 epochs while collecting validation labels and predictions.

```

In [18]: model = CarClassifierResNet(num_classes=num_classes).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad,model.parameters()),lr=0.001)

all_labels,all_predictions=train_model(model,criterion,optimizer,epochs=10)

```

C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\optim\adam.py:534: UserWarning: The operator 'aten::lerp.Scalar_out' is not currently supported on the DML backend and will fall back to run on the CPU. This may have performance implications. (Triggered internally at C:__w\1\s\pytorch-directml-plugin\torch_directml\csrc\dml\dml_cpu_fallback.cpp:17.)

torch._foreach_lerp_(device_exp_avgs, device_grads, 1 - betas)

```
Batch:10, Loss:0.78
Batch:20, Loss:0.77
Batch:30, Loss:0.74
Batch:40, Loss:0.67
Batch:50, Loss:0.52
Epoch:[1/10], Avg Loss:0.8269
*** Validation Accuracy: 73.57%***
Execution_time: 193.74677276611328 seconds
Batch:10, Loss:0.41
Batch:20, Loss:0.61
Batch:30, Loss:0.42
Batch:40, Loss:0.39
Batch:50, Loss:0.35
Epoch:[2/10], Avg Loss:0.3843
*** Validation Accuracy: 73.74%***
Execution_time: 366.20083379745483 seconds
Batch:10, Loss:0.13
Batch:20, Loss:0.31
Batch:30, Loss:0.32
Batch:40, Loss:0.30
Batch:50, Loss:0.13
Epoch:[3/10], Avg Loss:0.1769
*** Validation Accuracy: 69.74%***
Execution_time: 531.1101233959198 seconds
Batch:10, Loss:0.27
Batch:20, Loss:0.30
Batch:30, Loss:0.23
Batch:40, Loss:0.05
Batch:50, Loss:0.04
Epoch:[4/10], Avg Loss:0.1248
*** Validation Accuracy: 76.00%***
Execution_time: 694.2749588489532 seconds
Batch:10, Loss:0.05
Batch:20, Loss:0.05
Batch:30, Loss:0.03
Batch:40, Loss:0.08
Batch:50, Loss:0.01
Epoch:[5/10], Avg Loss:0.0635
*** Validation Accuracy: 75.65%***
Execution_time: 860.2879436016083 seconds
Batch:10, Loss:0.03
Batch:20, Loss:0.01
Batch:30, Loss:0.40
Batch:40, Loss:0.61
Batch:50, Loss:0.17
Epoch:[6/10], Avg Loss:0.0982
*** Validation Accuracy: 72.17%***
Execution_time: 1023.0844266414642 seconds
Batch:10, Loss:0.57
Batch:20, Loss:0.06
Batch:30, Loss:0.15
Batch:40, Loss:0.02
Batch:50, Loss:0.13
Epoch:[7/10], Avg Loss:0.1197
*** Validation Accuracy: 74.96%***
Execution_time: 1186.9372272491455 seconds
Batch:10, Loss:0.09
Batch:20, Loss:0.02
Batch:30, Loss:0.02
Batch:40, Loss:0.05
Batch:50, Loss:0.04
Epoch:[8/10], Avg Loss:0.0847
*** Validation Accuracy: 73.39%***
Execution_time: 1350.2097430229187 seconds
Batch:10, Loss:0.00
Batch:20, Loss:0.01
Batch:30, Loss:0.00
Batch:40, Loss:0.03
Batch:50, Loss:0.04
Epoch:[9/10], Avg Loss:0.0430
*** Validation Accuracy: 76.00%***
Execution_time: 1514.5076122283936 seconds
Batch:10, Loss:0.05
Batch:20, Loss:0.15
Batch:30, Loss:0.10
Batch:40, Loss:0.01
Batch:50, Loss:0.02
Epoch:[10/10], Avg Loss:0.0446
*** Validation Accuracy: 80.52%***
Execution_time: 1678.4959802627563 seconds
```

Model Evaluation using Confusion Matrix and Classification Report

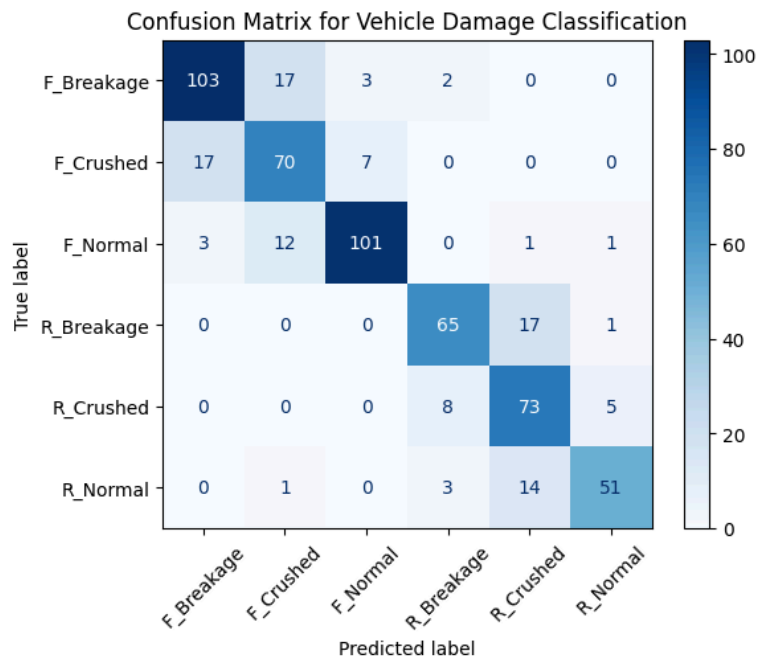
```
In [21]: from sklearn.metrics import classification_report

report = classification_report(all_labels, all_predictions)
print(report)
```

	precision	recall	f1-score	support
0	0.84	0.82	0.83	125
1	0.70	0.74	0.72	94
2	0.91	0.86	0.88	118
3	0.83	0.78	0.81	83
4	0.70	0.85	0.76	86
5	0.88	0.74	0.80	69
accuracy			0.81	575
macro avg	0.81	0.80	0.80	575
weighted avg	0.81	0.81	0.81	575

```
In [25]: import numpy as np
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from matplotlib import pyplot as plt

conf_matrix = confusion_matrix(all_labels, all_predictions, labels=np.arange(num_classes))
disp = ConfusionMatrixDisplay(confusion_matrix=conf_matrix, display_labels=class_names)
disp.plot(cmap=plt.cm.Blues, xticks_rotation=45)
plt.title("Confusion Matrix for Vehicle Damage Classification")
plt.show()
```



Save the model

```
In [27]: torch.save(model.state_dict(), 'saved_model.pth')
```

Model training with parameters after hyperparameter training in different notebook

```
In [19]: model = CarClassifierResNet(num_classes=num_classes, dropout_rate=0.6).to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=0.0005)

labels, predictions = train_model(model, criterion, optimizer, epochs=10)
```

```
Batch:10, Loss:1.29
Batch:20, Loss:0.99
Batch:30, Loss:0.81
Batch:40, Loss:0.80
Batch:50, Loss:0.67
Epoch:[1/10], Avg Loss:0.9384
*** Validation Accuracy: 72.00%***
Execution_time: 163.24656414985657 seconds
Batch:10, Loss:0.31
Batch:20, Loss:0.32
Batch:30, Loss:0.42
Batch:40, Loss:0.25
Batch:50, Loss:0.34
Epoch:[2/10], Avg Loss:0.3415
*** Validation Accuracy: 76.52%***
Execution_time: 327.94937658309937 seconds
Batch:10, Loss:0.15
Batch:20, Loss:0.27
Batch:30, Loss:0.19
Batch:40, Loss:0.07
Batch:50, Loss:0.05
Epoch:[3/10], Avg Loss:0.1559
*** Validation Accuracy: 72.52%***
Execution_time: 492.4220983982086 seconds
Batch:10, Loss:0.57
Batch:20, Loss:0.14
Batch:30, Loss:0.16
Batch:40, Loss:0.05
Batch:50, Loss:0.02
Epoch:[4/10], Avg Loss:0.1193
*** Validation Accuracy: 74.09%***
Execution_time: 656.2942147254944 seconds
Batch:10, Loss:0.04
Batch:20, Loss:0.06
Batch:30, Loss:0.02
Batch:40, Loss:0.14
Batch:50, Loss:0.06
Epoch:[5/10], Avg Loss:0.0922
*** Validation Accuracy: 75.65%***
Execution_time: 820.2054462432861 seconds
Batch:10, Loss:0.01
Batch:20, Loss:0.09
Batch:30, Loss:0.05
Batch:40, Loss:0.11
Batch:50, Loss:0.09
Epoch:[6/10], Avg Loss:0.0749
*** Validation Accuracy: 75.65%***
Execution_time: 985.822408914566 seconds
Batch:10, Loss:0.02
Batch:20, Loss:0.05
Batch:30, Loss:0.01
Batch:40, Loss:0.07
Batch:50, Loss:0.01
Epoch:[7/10], Avg Loss:0.0394
*** Validation Accuracy: 74.61%***
Execution_time: 1151.5265464782715 seconds
Batch:10, Loss:0.01
Batch:20, Loss:0.00
Batch:30, Loss:0.01
Batch:40, Loss:0.00
Batch:50, Loss:0.08
Epoch:[8/10], Avg Loss:0.0342
*** Validation Accuracy: 75.65%***
Execution_time: 1316.0039563179016 seconds
Batch:10, Loss:0.04
Batch:20, Loss:0.01
Batch:30, Loss:0.00
Batch:40, Loss:0.04
Batch:50, Loss:0.00
Epoch:[9/10], Avg Loss:0.0140
*** Validation Accuracy: 77.04%***
Execution_time: 1480.6483693122864 seconds
Batch:10, Loss:0.00
Batch:20, Loss:0.04
Batch:30, Loss:0.00
Batch:40, Loss:0.00
Batch:50, Loss:0.03
Epoch:[10/10], Avg Loss:0.0147
*** Validation Accuracy: 76.52%***
Execution_time: 1644.996248960495 seconds
```

Model Evaluation using Confusion Matrix and Classification Report

```
In [22]: from sklearn.metrics import classification_report

report = classification_report(labels, predictions)
print(report)
```

	precision	recall	f1-score	support
0	0.79	0.87	0.83	125
1	0.71	0.63	0.67	94
2	0.84	0.82	0.83	118
3	0.82	0.72	0.77	83
4	0.68	0.70	0.69	86
5	0.71	0.80	0.75	69
accuracy			0.77	575
macro avg	0.76	0.76	0.76	575
weighted avg	0.77	0.77	0.76	575

Due to hardware and device constraints, extensive hyperparameter tuning could not be performed. Initial tuning experiments resulted in lower validation accuracy than the ResNet-50 transfer learning model. Since ResNet-50 consistently achieved the best performance, it was selected as the final model for this project.



Vehicle Damage Detection: Hyperparameter Tuning

```
In [20]: import torch
import torch_directml
import pandas as pd
from sklearn.model_selection import train_test_split
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from matplotlib import pyplot as plt
from torchvision import datasets, transforms
import torchvision
from torchvision import models
from torchvision.models import efficientnet_b0, EfficientNet_B0_Weights
from torch.nn import BatchNorm2d
import torch.nn.functional as F
import optuna
import time
import os
```

```
In [3]: device = torch_directml.device()
device
```

```
Out[3]: device(type='privateuseone', index=0)
```

Load Data

```
In [4]: image_transforms=transforms.Compose([
    transforms.RandomHorizontalFlip(10),
    transforms.ColorJitter(brightness=0.2,contrast=0.2),
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485,0.456,0.406],std=[0.229,0.224,0.225])
])
```

```
In [5]: dataset_path = "./dataset"

dataset = datasets.ImageFolder(root=dataset_path, transform=image_transforms)
len(dataset)
```

```
Out[5]: 2300
```

```
In [6]: 2300*0.75
```

```
Out[6]: 1725.0
```

```
In [7]: dataset_path = "./dataset"

dataset = datasets.ImageFolder(root=dataset_path,transform=image_transforms)
len(dataset)
```

```
Out[7]: 2300
```

```
In [8]: dataset.classes
```

```
Out[8]: ['F_Breakage', 'F_Crushed', 'F_Normal', 'R_Breakage', 'R_Crushed', 'R_Normal']
```

```
In [9]: num_classes = len(dataset.classes)
num_classes
```

```
Out[9]: 6
```

```
In [11]: train_size = int(0.75*len(dataset))
val_size = len(dataset)-train_size
train_size,val_size
```

```
Out[11]: (1725, 575)
```

```
In [12]: from torch.utils.data import random_split

train_dataset,val_dataset = random_split(dataset,[train_size,val_size])
```

```
In [13]: train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=True)
```

```
In [14]: for images,labels in train_loader:
    print(images.shape)
    print(labels.shape)
    break
```

```
torch.Size([32, 3, 224, 224])
torch.Size([32])
```

```
In [14]: labels[0]
```

```
Out[14]: tensor(0)
```

```
In [15]: images[0].shape
```

```
Out[15]: torch.Size([3, 224, 224])
```



```
In [15]: #reshaping to plot using imshow
images[0].permute(1,2,0).shape
```

```
Out[15]: torch.Size([224, 224, 3])
```

```
In [16]: #reshaping to plot using imshow
images[0].permute(1,2,0).shape
```

```
Out[16]: torch.Size([224, 224, 3])
```

Model Training & Hyperparameter Tunning

Define a ResNet-50 transfer learning model by loading pretrained ImageNet weights, freezing most layers, fine-tuning the final feature extraction block (layer4), and replacing the classification head with a customizable dropout layer and fully connected layer for hyperparameter tuning.

```
In [17]: # Load the pre-trained ResNet model
class CarClassifierResNet(nn.Module):
    def __init__(self, num_classes, dropout_rate=0.5):
        super().__init__()
        self.model = models.resnet50(weights='DEFAULT')
        # Freeze all layers except the final fully connected layer
        for param in self.model.parameters():
            param.requires_grad = False

        # Unfreeze layer4 and fc layers
        for param in self.model.layer4.parameters():
            param.requires_grad = True

        # Replace the final fully connected layer
        self.model.fc = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(self.model.fc.in_features, num_classes)
        )

    def forward(self, x):
        x = self.model(x)
        return x
```

Define the Optuna objective function to automatically optimize the ResNet-50 model by searching for the best combination of hyperparameters, including learning rate, dropout rate, weight decay, optimizer, batch size, and training epochs, based on validation accuracy.

```
In [23]: # Define the objective function for Optuna
def objective(trial):
    # Suggest values for the hyperparameters
    lr = trial.suggest_float('lr', 1e-5, 1e-2, log=True)
    dropout_rate = trial.suggest_float('dropout_rate', 0.2, 0.7)
    weight_decay = trial.suggest_float('weight_decay', 1e-5, 1e-2, log=True)
    optimizer_name = trial.suggest_categorical('optimizer', ['SGD', 'Adam', 'RMSprop'])
    batch_size = trial.suggest_categorical('batch_size', [32, 64, 128])
    num_epochs = trial.suggest_int('num_epochs', 5, 15)

    # Load the model
    model = CarClassifierResNet(num_classes=num_classes, dropout_rate=dropout_rate).to(device)

    # data
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

    #optimizer
    if optimizer_name == 'SGD':
        optimizer = optim.SGD(model.parameters(), lr=lr, weight_decay=weight_decay)
    elif optimizer_name == 'Adam':
        optimizer = optim.Adam(model.parameters(), lr=lr, weight_decay=weight_decay)
    elif optimizer_name == 'RMSprop':
        optimizer = optim.RMSprop(model.parameters(), lr=lr, weight_decay=weight_decay)

    # Define the Loss function and optimizer
    criterion = nn.CrossEntropyLoss()

    # Training Loop (using fewer epochs for faster hyperparameter tuning)
    start = time.time()
    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0
        for batch_num, (images, labels) in enumerate(train_loader):
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * images.size(0)

        epoch_loss = running_loss / len(train_loader.dataset)

    # Validation Loop
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
```

```

_, predicted = torch.max(outputs.data, 1)
total += labels.size(0)
correct += (predicted == labels).sum().item()

accuracy = 100 * correct / total

# Report intermediate result to Optuna
trial.report(accuracy, epoch)

# Handle pruning (if applicable)
if trial.should_prune():
    raise optuna.exceptions.TrialPruned()

end = time.time()
print(f"Execution time: {end - start} seconds")

return accuracy

```

```

In [24]: # Create the study and optimize
import optuna

pruner=optuna.pruners.MedianPruner()
study = optuna.create_study(direction='maximize',pruner=pruner)
study.optimize(objective, n_trials=20)

```

```

[I 2026-06-21 16:24:10,212] A new study created in memory with name: no-name-941eda64-ef61-44f3-90d8-f0f00f1c12a1
[I 2026-06-21 16:42:18,658] Trial 0 finished with value: 77.91304347826087 and parameters: {'lr': 3.6199213500635116e-05, 'dropout_rate': 0.23005950635614947, 'weight_decay': 0.00027821858013912294, 'optimizer': 'RMSprop', 'batch_size': 32, 'num_epochs': 6}. Best is trial 0 with value: 77.91304347826087.
Execution time: 1087.598884344101 seconds

```

```

[W 2026-06-21 16:59:12,990] Trial 1 failed with parameters: {'lr': 0.0002796324107385374, 'dropout_rate': 0.4151938100081717, 'weight_decay': 0.007968513179857192, 'optimizer': 'RMSprop', 'batch_size': 128, 'num_epochs': 11} because of the following error: RuntimeError('Could not allocate tensor with 102760448 bytes. There is not enough GPU video memory available!').

```

Traceback (most recent call last):

```

File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\optuna\study\_optimize.py", line 206, in _run_trial
    value_or_values = func(trial)
File "C:\Users\Tanmay\AppData\Local\Temp\ipykernel_31332\3523705390.py", line 40, in objective
    outputs = model(images)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1553, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1562, in _call_impl
    return forward_call(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Temp\ipykernel_31332\756172557.py", line 21, in forward
    x = self.model(x)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1553, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1562, in _call_impl
    return forward_call(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torchvision\models\resnet.py", line 285, in forward
    return self._forward_impl(x)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torchvision\models\resnet.py", line 275, in _forward_impl
    x = self.layer3(x)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1553, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1562, in _call_impl
    return forward_call(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\container.py", line 219, in forward
    input = module(input)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1553, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1562, in _call_impl
    return forward_call(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torchvision\models\resnet.py", line 155, in forward
    out = self.bn3(out)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1553, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py", line 1562, in _call_impl
    return forward_call(*args, **kwargs)
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\batchnorm.py", line 176, in forward
    return F.batch_norm(
File "C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\functional.py", line 2512, in batch_norm
    return torch.batch_norm(

```

RuntimeError: Could not allocate tensor with 102760448 bytes. There is not enough GPU video memory available!

```

[W 2026-06-21 16:59:13,522] Trial 1 failed with value None.

```

```

RuntimeError                                Traceback (most recent call last)
Cell In[24], line 6
      4 pruner=optuna.pruners.MedianPruner()
      5 study = optuna.create_study(direction='maximize',pruner=pruner)
----> 6 study.optimize(objective, n_trials=20)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\optuna\study\study.py:490, in Study.optimize(self, func, n_trials, timeout, n_jobs, catch, callbacks, gc_after_trial, show_progress_bar)
    388 def optimize(
    389     self,
    390     func: ObjectiveFuncType,
    (...)
    397     show_progress_bar: bool = False,
    398 ) -> None:
    399     """Optimize an objective function.
    400
    401     Optimization is done by choosing a suitable set of hyperparameter values from a given
    (...)
    488
    489     """
    490     _optimize(
    491         study=self,
    492         func=func,
    493         n_trials=n_trials,
    494         timeout=timeout,
    495         n_jobs=n_jobs,
    496         catch=tuple(catch) if isinstance(catch, Iterable) else (catch,),
    497         callbacks=callbacks,
    498         gc_after_trial=gc_after_trial,
    499         show_progress_bar=show_progress_bar,
    500     )

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\optuna\study\_optimize.py:68, in _optimize(study, func, n_trials, timeout, n_jobs, catch, callbacks, gc_after_trial, show_progress_bar)
     66 try:
     67     if n_jobs == 1:
----> 68     _optimize_sequential(
     69         study,
     70         func,
     71         n_trials,
     72         timeout,
     73         catch,
     74         callbacks,
     75         gc_after_trial,
     76         reseed_sampler_rng=False,
     77         time_start=None,
     78         progress_bar=progress_bar,
     79     )
     80 else:
     81     if n_jobs == -1:

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\optuna\study\_optimize.py:165, in _optimize_sequential(study, func, n_trials, timeout, catch, callbacks, gc_after_trial, reseed_sampler_rng, time_start, progress_bar)
    162     break
    164 try:
--> 165     frozen_trial_id = _run_trial(study, func, catch)
    166 finally:
    167     # The following line mitigates memory problems that can be occurred in some
    168     # environments (e.g., services that use computing containers such as GitHub Actions).
    169     # Please refer to the following PR for further details:
    170     # https://github.com/optuna/optuna/pull/325.
    171     if gc_after_trial:

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\optuna\study\_optimize.py:263, in _run_trial(study, func, catch)
    256     assert False, "Should not reach."
    258 if (
    259     updated_state == TrialState.FAIL
    260     and func_err is not None
    261     and not isinstance(func_err, catch)
    262 ):
--> 263     raise func_err
    264 return trial._trial_id

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\optuna\study\_optimize.py:206, in _run_trial(study, func, catch)
    204 with get_heartbeat_thread(trial._trial_id, study._storage):
    205     try:
--> 206     value_or_values = func(trial)
    207     except exceptions.TrialPruned as e:
    208         # TODO(mamu): Handle multi-objective cases.
    209         state = TrialState.PRUNED

Cell In[23], line 40, in objective(trial)
     37 images, labels = images.to(device), labels.to(device)
     39 optimizer.zero_grad()
--> 40 outputs = model(images)
     41 loss = criterion(outputs, labels)
     42 loss.backward()

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1553, in Module._wrapped_call_impl(self, *args, **kwargs)
    1551 return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1552 else:
-> 1553     return self._call_impl(*args, **kwargs)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1562, in Module._call_impl(self, *args, **kwargs)
    1557 # If we don't have any hooks, we want to skip the rest of the logic in
    1558 # this function, and just call forward.
    1559 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1560         or _global_backward_pre_hooks or _global_backward_hooks

```

```

1561         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1562     return forward_call(*args, **kwargs)
1564 try:
1565     result = None

Cell In[18], line 21, in CarClassifierResNet.forward(self, x)
    20 def forward(self, x):
--> 21     x = self.model(x)
    22     return x

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1553, in Module._wrapped_call_impl(self, *args, **kwargs)
    1551     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1552 else:
-> 1553     return self._call_impl(*args, **kwargs)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1562, in Module._call_impl(self, *args, **kwargs)
    1557 # If we don't have any hooks, we want to skip the rest of the logic in
    1558 # this function, and just call forward.
    1559 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1560         or _global_backward_pre_hooks or _global_backward_hooks
    1561         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1562     return forward_call(*args, **kwargs)
    1564 try:
    1565     result = None

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torchvision\models\resnet.py:285, in ResNet.forward(self, x)
    284 def forward(self, x: Tensor) -> Tensor:
-> 285     return self._forward_impl(x)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torchvision\models\resnet.py:275, in ResNet._forward_impl(self, x)
    273 x = self.layer1(x)
    274 x = self.layer2(x)
--> 275 x = self.layer3(x)
    276 x = self.layer4(x)
    277 x = self.avgpool(x)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1553, in Module._wrapped_call_impl(self, *args, **kwargs)
    1551     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1552 else:
-> 1553     return self._call_impl(*args, **kwargs)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1562, in Module._call_impl(self, *args, **kwargs)
    1557 # If we don't have any hooks, we want to skip the rest of the logic in
    1558 # this function, and just call forward.
    1559 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1560         or _global_backward_pre_hooks or _global_backward_hooks
    1561         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1562     return forward_call(*args, **kwargs)
    1564 try:
    1565     result = None

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\container.py:219, in Sequential.forward(self, input)
    217 def forward(self, input):
    218     for module in self:
--> 219         input = module(input)
    220     return input

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1553, in Module._wrapped_call_impl(self, *args, **kwargs)
    1551     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1552 else:
-> 1553     return self._call_impl(*args, **kwargs)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1562, in Module._call_impl(self, *args, **kwargs)
    1557 # If we don't have any hooks, we want to skip the rest of the logic in
    1558 # this function, and just call forward.
    1559 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1560         or _global_backward_pre_hooks or _global_backward_hooks
    1561         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1562     return forward_call(*args, **kwargs)
    1564 try:
    1565     result = None

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torchvision\models\resnet.py:155, in Bottleneck.forward(self, x)
    152 out = self.relu(out)
    154 out = self.conv3(out)
--> 155 out = self.bn3(out)
    157 if self.downsample is not None:
    158     identity = self.downsample(x)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1553, in Module._wrapped_call_impl(self, *args, **kwargs)
    1551     return self._compiled_call_impl(*args, **kwargs) # type: ignore[misc]
    1552 else:
-> 1553     return self._call_impl(*args, **kwargs)

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\module.py:1562, in Module._call_impl(self, *args, **kwargs)
    1557 # If we don't have any hooks, we want to skip the rest of the logic in
    1558 # this function, and just call forward.
    1559 if not (self._backward_hooks or self._backward_pre_hooks or self._forward_hooks or self._forward_pre_hooks
    1560         or _global_backward_pre_hooks or _global_backward_hooks
    1561         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1562     return forward_call(*args, **kwargs)
    1564 try:
    1565     result = None

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\modules\batchnorm.py:176, in _BatchNorm.forward(self, input)
    169     bn_training = (self.running_mean is None) and (self.running_var is None)
    171 r"""
    172 Buffers are only updated if they are to be tracked and we are in training mode. Thus they only need to be
    173 passed when the update should occur (i.e. in training mode when they are tracked), or when buffer stats are
    174 used for normalization (i.e. in eval mode when buffers are not None).

```

```

175 """
--> 176 return F.batch_norm(
177     input,
178     # If buffers are not to be tracked, ensure that they won't be updated
179     self.running_mean
180     if not self.training or self.track_running_stats
181     else None,
182     self.running_var if not self.training or self.track_running_stats else None,
183     self.weight,
184     self.bias,
185     bn_training,
186     exponential_average_factor,
187     self.eps,
188 )

```

File ~\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\nn\functional.py:2512, in batch_norm(input, running_mean, running_var, weight, bias, training, momentum, eps)

```

2509 if training:
2510     _verify_batch_size(input.size())
-> 2512 return torch.batch_norm(
2513     input, weight, bias, running_mean, running_var, training, momentum, eps, torch.backends.cudnn.enabled
2514 )

```

RuntimeError: Could not allocate tensor with 102760448 bytes. There is not enough GPU video memory available!

Note: Due to limited GPU (DirectML) memory, extensive hyperparameter tuning with larger batch sizes and multiple trials resulted in "RuntimeError: Could not allocate tensor...". Consequently, the hyperparameter search was constrained, and the final model selection was based on the best-performing configuration that could be trained within the available hardware resources.

```
In [25]: print(study.best_params)
print(study.best_value)
```

```
{'lr': 3.6199213500635116e-05, 'dropout_rate': 0.23005950635614947, 'weight_decay': 0.00027821858013912294, 'optimizer': 'RMSprop', 'batch_size': 32, 'num_epochs': 6}
77.91304347826087
```

Define the Optuna objective function to optimize the ResNet-50 model by tuning key hyperparameters while incorporating memory management and exception handling to prevent GPU out-of-memory errors during trials on limited DirectML hardware.

```
In [27]: import gc

def objective(trial):
    lr = trial.suggest_float('lr', 1e-5, 1e-2, log=True)
    dropout_rate = trial.suggest_float('dropout_rate', 0.2, 0.7)
    weight_decay = trial.suggest_float('weight_decay', 1e-5, 1e-2, log=True)
    optimizer_name = trial.suggest_categorical('optimizer', ['SGD', 'Adam', 'RMSprop'])
    batch_size = trial.suggest_categorical('batch_size', [16, 32, 64]) # 128 removed - OOMs on integrated GPU
    num_epochs = trial.suggest_int('num_epochs', 3, 8)

    model = None
    try:
        model = CarClassifierResNet(num_classes=num_classes, dropout_rate=dropout_rate).to(device)

        train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
        val_loader = DataLoader(val_dataset, batch_size=batch_size, shuffle=False)

        trainable_params = filter(lambda p: p.requires_grad, model.parameters())
        if optimizer_name == 'SGD':
            optimizer = optim.SGD(trainable_params, lr=lr, weight_decay=weight_decay)
        elif optimizer_name == 'Adam':
            optimizer = optim.Adam(trainable_params, lr=lr, weight_decay=weight_decay)
        elif optimizer_name == 'RMSprop':
            optimizer = optim.RMSprop(trainable_params, lr=lr, weight_decay=weight_decay)

        criterion = nn.CrossEntropyLoss()

        start = time.time()
        accuracy = 0.0
        for epoch in range(num_epochs):
            model.train()
            running_loss = 0.0
            for batch_num, (images, labels) in enumerate(train_loader):
                images, labels = images.to(device), labels.to(device)
                optimizer.zero_grad()
                outputs = model(images)
                loss = criterion(outputs, labels)
                loss.backward()
                optimizer.step()
                running_loss += loss.item() * images.size(0)
            epoch_loss = running_loss / len(train_loader.dataset)

            model.eval()
            correct = 0
            total = 0
            with torch.no_grad():
                for images, labels in val_loader:
                    images, labels = images.to(device), labels.to(device)
                    outputs = model(images)
                    _, predicted = torch.max(outputs.data, 1)
                    total += labels.size(0)
                    correct += (predicted == labels).sum().item()

            accuracy = 100 * correct / total
            trial.report(accuracy, epoch)
            if trial.should_prune():
                raise optuna.exceptions.TrialPruned()

```

```

end = time.time()
print(f"Trial {trial.number} done in {end - start:.1f}s | "
      f"batch_size={batch_size}, lr={lr:.2e}, optimizer={optimizer_name}, acc={accuracy:.2f}%")
return accuracy

except RuntimeError as e:
    if "memory" in str(e).lower() or "allocate" in str(e).lower():
        print(f"Trial {trial.number} OOM (batch_size={batch_size}) - pruning this trial.")
        raise optuna.exceptions.TrialPruned()
    raise # Let genuine bugs surface instead of silently swallowing them

finally:
    del model
    gc.collect() # main memory-reclaim lever available on DirectML/integrated GPU

```

Create an Optuna study with a Median Pruner to maximize validation accuracy and efficiently optimize hyperparameters by pruning underperforming or memory-constrained trials.

Note: This Optuna optimization step was not executed because the available hardware (integrated GPU with DirectML) was unable to support multiple hyperparameter tuning trials due to GPU memory limitations and long execution times.

```

In [ ]: pruner = optuna.pruners.MedianPruner()
study = optuna.create_study(direction='maximize', pruner=pruner)
study.optimize(objective, n_trials=20, catch=(RuntimeError,))

```

[I 2026-06-21 17:08:38,159] A new study created in memory with name: no-name-160802fb-6b81-4d1c-8bdd-9e0d17d3b102

Note: Hyperparameter tuning was performed using Optuna by optimizing the learning rate and dropout rate. To accommodate the available hardware resources, the search space and training epochs were intentionally limited, allowing the optimization to complete within the device's computational constraints.

```

In [18]: # Define the objective function for Optuna
def objective(trial):
    # Suggest values for the hyperparameters
    lr = trial.suggest_float('lr', 1e-5, 1e-2, log=True)
    dropout_rate = trial.suggest_float('dropout_rate', 0.2, 0.7)

    # Load the model
    model = CarClassifierResNet(num_classes=num_classes, dropout_rate=dropout_rate).to(device)

    # Define the loss function and optimizer
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(filter(lambda p: p.requires_grad, model.parameters()), lr=lr)

    # Training Loop (using fewer epochs for faster hyperparameter tuning)
    epochs = 3
    start = time.time()
    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        for batch_num, (images, labels) in enumerate(train_loader):
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item() * images.size(0)

        epoch_loss = running_loss / len(train_loader.dataset)

    # Validation Loop
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for images, labels in val_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100 * correct / total

    # Report intermediate result to Optuna
    trial.report(accuracy, epoch)

    # Handle pruning (if applicable)
    if trial.should_prune():
        raise optuna.exceptions.TrialPruned()

    end = time.time()
    print(f"Execution time: {end - start} seconds")

    return accuracy

```

Create an Optuna study to maximize validation accuracy and execute 20 hyperparameter optimization trials using the defined objective function.

```

In [21]: # Create the study and optimize
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)

```

```
[I 2026-06-21 22:11:03,646] A new study created in memory with name: no-name-ce05d290-8440-430b-93eb-70321f6fbc33
C:\Users\Tanmay\AppData\Local\Programs\Python\Python310\lib\site-packages\torch\optim\adam.py:534: UserWarning: The operator 'aten::lerp.Scalar_out' is
not currently supported on the DML backend and will fall back to run on the CPU. This may have performance implications. (Triggered internally at C:\_
w\1\s\pytorch-directml-plugin\torch_directml\csrc\dml\dml_cpu_fallback.cpp:17.)
  torch._foreach_lerp(device_exp_avgs, device_grads, 1 - beta1)
[I 2026-06-21 22:22:34,131] Trial 0 finished with value: 76.52173913043478 and parameters: {'lr': 0.0009307559115742382, 'dropout_rate': 0.288339020307
1097}. Best is trial 0 with value: 76.52173913043478.
Execution time: 689.3257238864899 seconds

[I 2026-06-21 22:33:48,582] Trial 1 finished with value: 78.6086956521739 and parameters: {'lr': 0.0001800358461726264, 'dropout_rate': 0.5065052855429
648}. Best is trial 1 with value: 78.6086956521739.
Execution time: 673.76078748703 seconds

[I 2026-06-21 22:45:01,867] Trial 2 finished with value: 67.82608695652173 and parameters: {'lr': 3.121226131872334e-05, 'dropout_rate': 0.571471677197
069}. Best is trial 1 with value: 78.6086956521739.
Execution time: 672.7268493175507 seconds

[I 2026-06-21 22:56:19,078] Trial 3 finished with value: 78.95652173913044 and parameters: {'lr': 0.0007963041308837038, 'dropout_rate': 0.254036001934
11957}. Best is trial 3 with value: 78.95652173913044.
Execution time: 676.6458880901337 seconds

[I 2026-06-21 23:07:48,618] Trial 4 finished with value: 77.21739130434783 and parameters: {'lr': 0.00023050722107025698, 'dropout_rate': 0.46950452780
65428}. Best is trial 3 with value: 78.95652173913044.
Execution time: 688.9391877651215 seconds

[I 2026-06-21 23:15:31,775] Trial 5 pruned.
[I 2026-06-21 23:19:14,701] Trial 6 pruned.
[I 2026-06-21 23:26:46,698] Trial 7 pruned.
[I 2026-06-21 23:35:40,386] Trial 8 finished with value: 75.47826086956522 and parameters: {'lr': 0.004566786733370688, 'dropout_rate': 0.5317489348922
233}. Best is trial 3 with value: 78.95652173913044.
Execution time: 533.1355192661285 seconds

[I 2026-06-21 23:38:21,134] Trial 9 pruned.
[I 2026-06-21 23:43:46,491] Trial 10 pruned.
[I 2026-06-21 23:49:09,667] Trial 11 pruned.
[I 2026-06-21 23:54:32,215] Trial 12 pruned.
[I 2026-06-21 23:57:12,268] Trial 13 pruned.
[I 2026-06-22 00:05:19,135] Trial 14 finished with value: 80.17391304347827 and parameters: {'lr': 0.0004323348992053245, 'dropout_rate': 0.60364993847
58081}. Best is trial 14 with value: 80.17391304347827.
Execution time: 486.5217316150665 seconds

[I 2026-06-22 00:08:02,235] Trial 15 pruned.
[I 2026-06-22 00:16:07,864] Trial 16 finished with value: 76.34782608695652 and parameters: {'lr': 0.0004710621845817836, 'dropout_rate': 0.59117534822
51626}. Best is trial 14 with value: 80.17391304347827.
Execution time: 485.2706437110901 seconds

[I 2026-06-22 00:18:49,768] Trial 17 pruned.
[I 2026-06-22 00:21:32,857] Trial 18 pruned.
[I 2026-06-22 00:26:57,371] Trial 19 pruned.
```

Display the best hyperparameter configuration found by Optuna along with its corresponding highest validation accuracy.

```
In [22]: print(study.best_params)
print(study.best_value)

{'lr': 0.0004323348992053245, 'dropout_rate': 0.6036499384758081}
80.17391304347827
```